

Сверточные нейронные сети для задачи сегментации



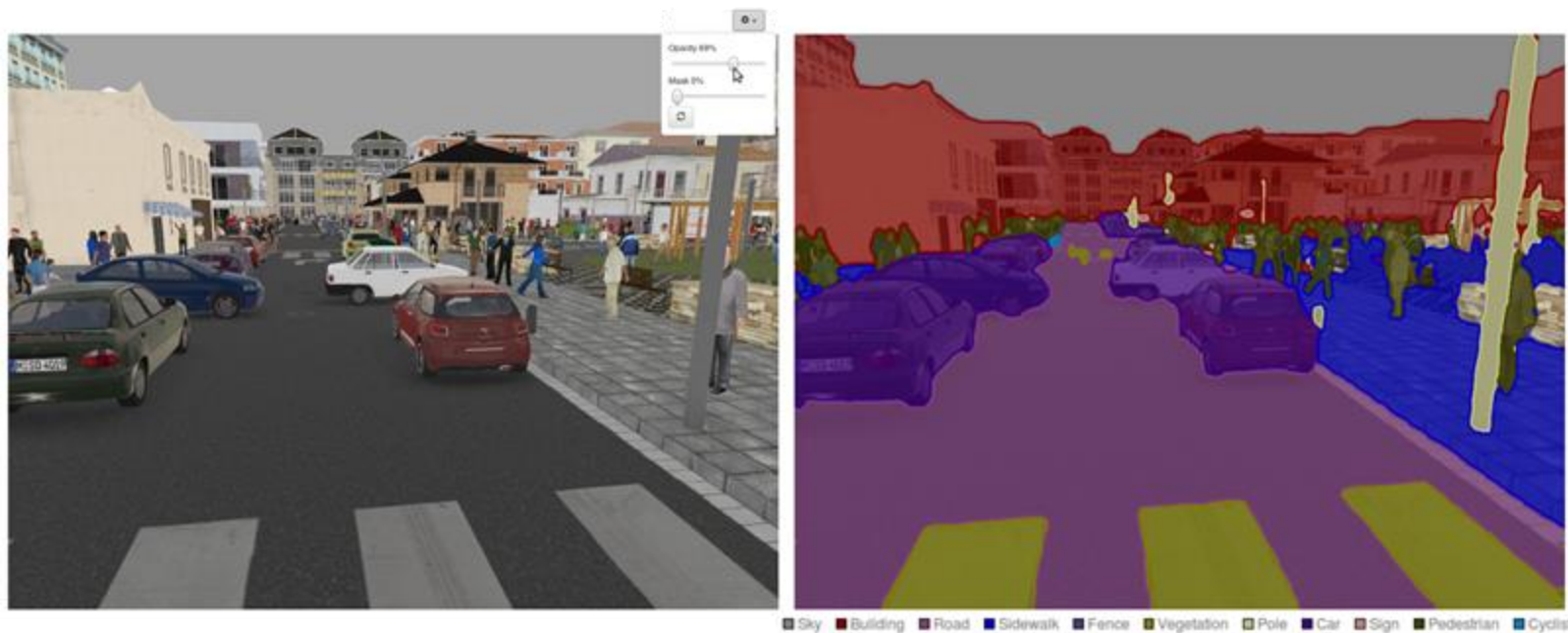
План лекции

- Постановка задачи сегментации изображений
- Базовые подходы к построению архитектур encoder-decoder
- Некоторые техники обучения моделей

Основные направления в CV

Сегментация

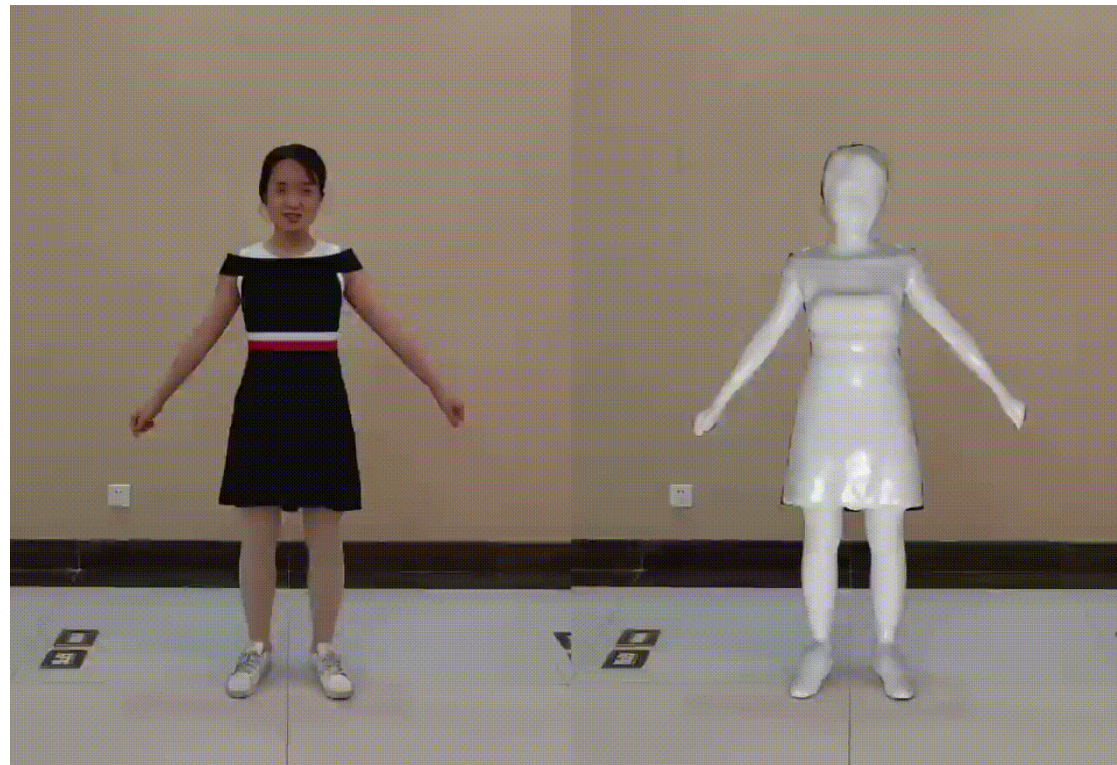
Решается задача классификации, но попиксельной



Основные направления в CV

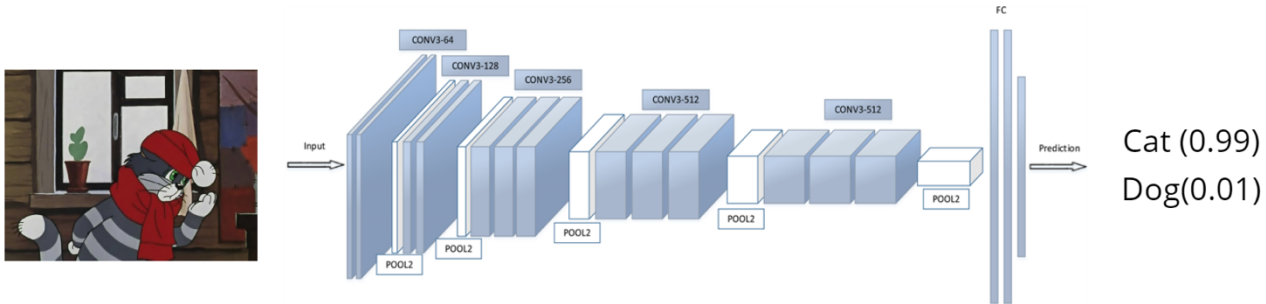
Построение 3D Объектов

Решается задача построения объемного объекта (obj-файл) по двумерному изображению



Классификация

Классификация изображения



Классификация пикселей изображения



- Сегментация — попиксельная классификация
- Не требует большого количества данных
- Все сегментационные модели — это архитектуры вида FCN

Типы сегментации

Semantic segmentation



segmented →

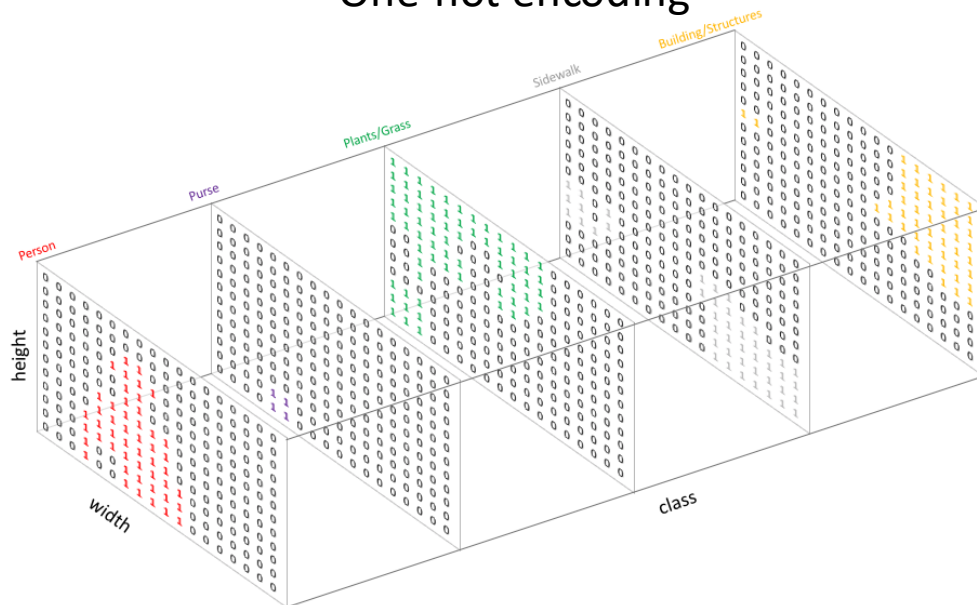
- 1: Person
- 2: Purse
- 3: Plants/Grass
- 4: Sidewalk
- 5: Building/Structures

3	3	3	3	3	3	3	3	3	3	3	3	3	5	5	5	5	5
3	3	3	3	3	3	3	3	3	3	3	3	3	5	5	5	5	5
3	3	3	3	3	3	1	1	3	3	3	3	5	5	5	5	5	5
3	3	3	3	3	1	1	1	1	3	3	5	5	5	5	5	5	5
3	3	3	3	3	1	1	1	3	3	5	5	5	5	5	5	5	5
5	5	3	3	3	1	1	3	3	5	5	5	5	5	5	5	5	5
4	4	3	4	1	1	1	1	1	4	4	4	5	5	5	5	5	5
4	4	3	4	1	1	1	1	1	4	4	4	4	4	5	5	5	5
4	4	4	1	1	1	1	1	1	1	4	4	4	4	4	4	4	4
3	3	3	1	1	1	1	1	1	1	4	4	4	4	4	4	4	4
3	3	3	1	2	2	1	1	1	1	4	4	4	4	4	4	4	4
3	3	3	1	2	1	1	1	1	1	4	4	4	4	4	4	4	4

Input

One-hot encoding

Semantic Labels

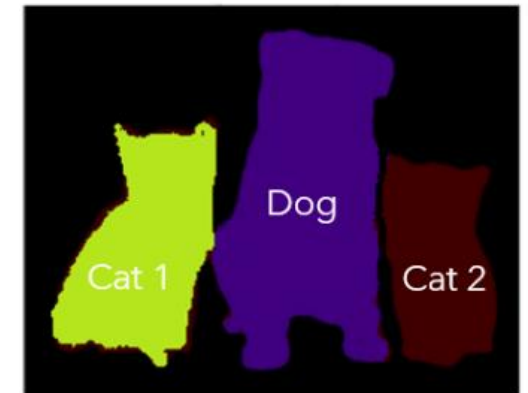


Instance segmentation

Original Image

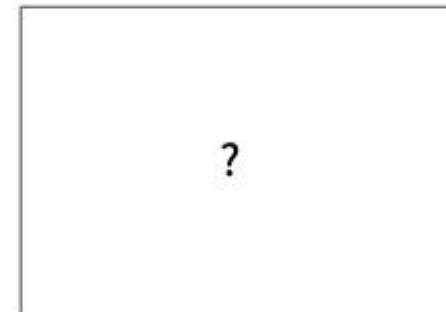


Instance Segmentation



Подходы сегментации

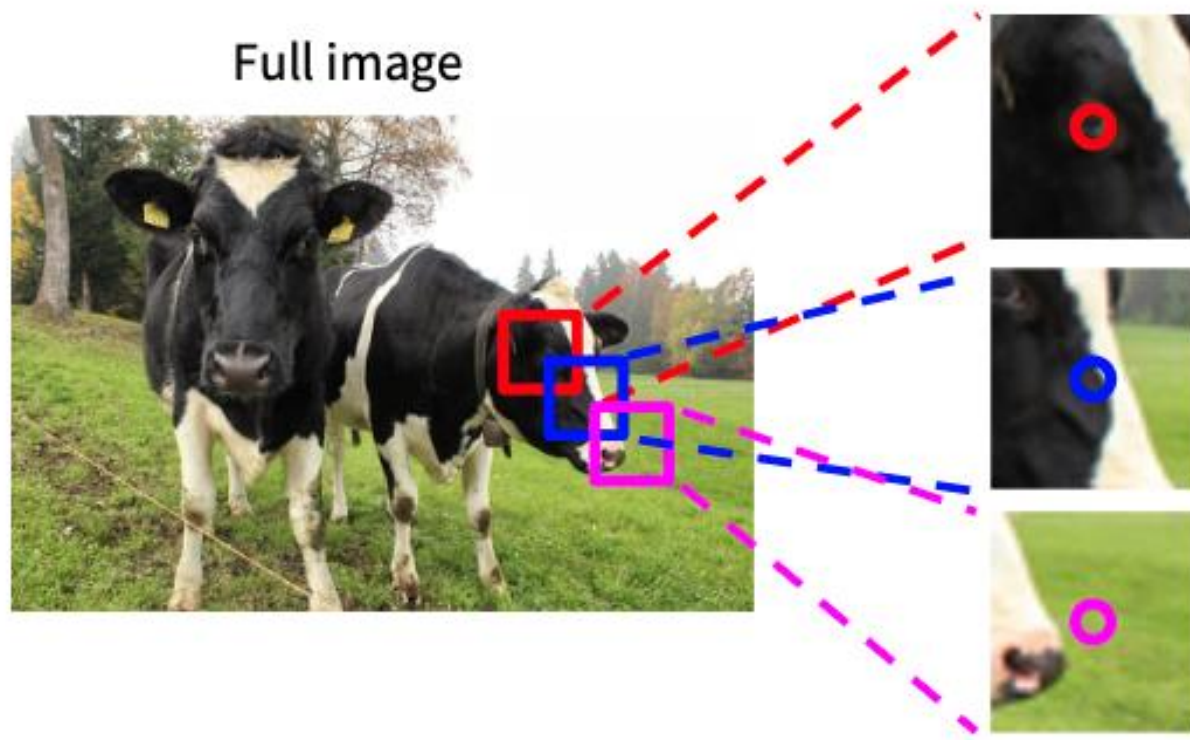
Semantic segmentation



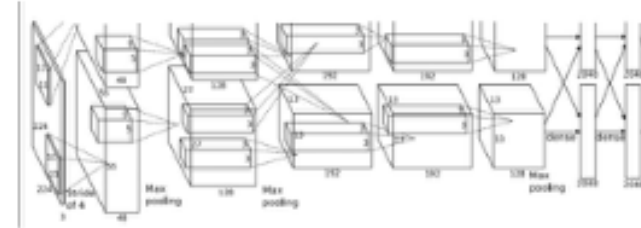
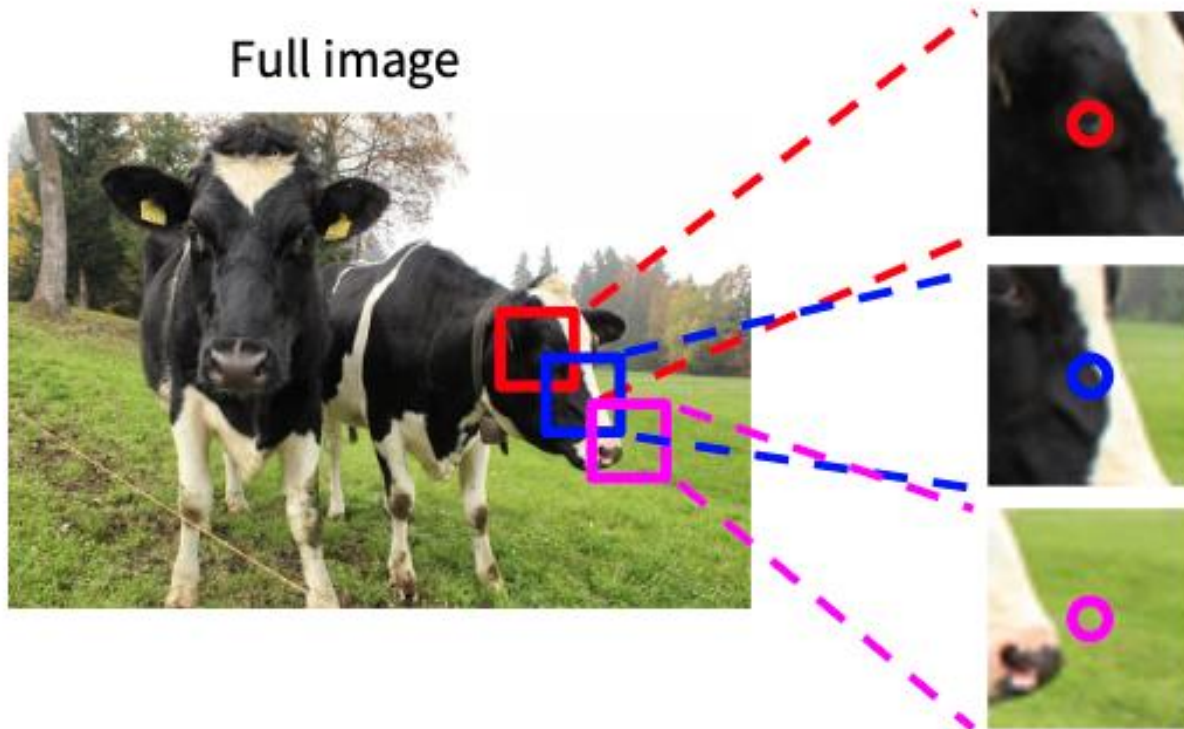
Подходы сегментации



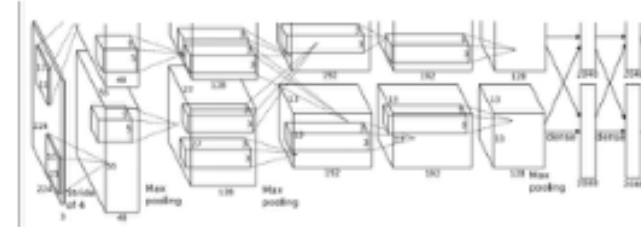
Подходы сегментации



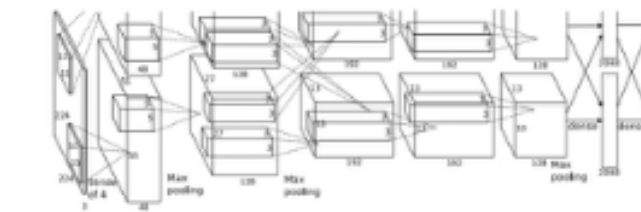
Подходы сегментации



Cow

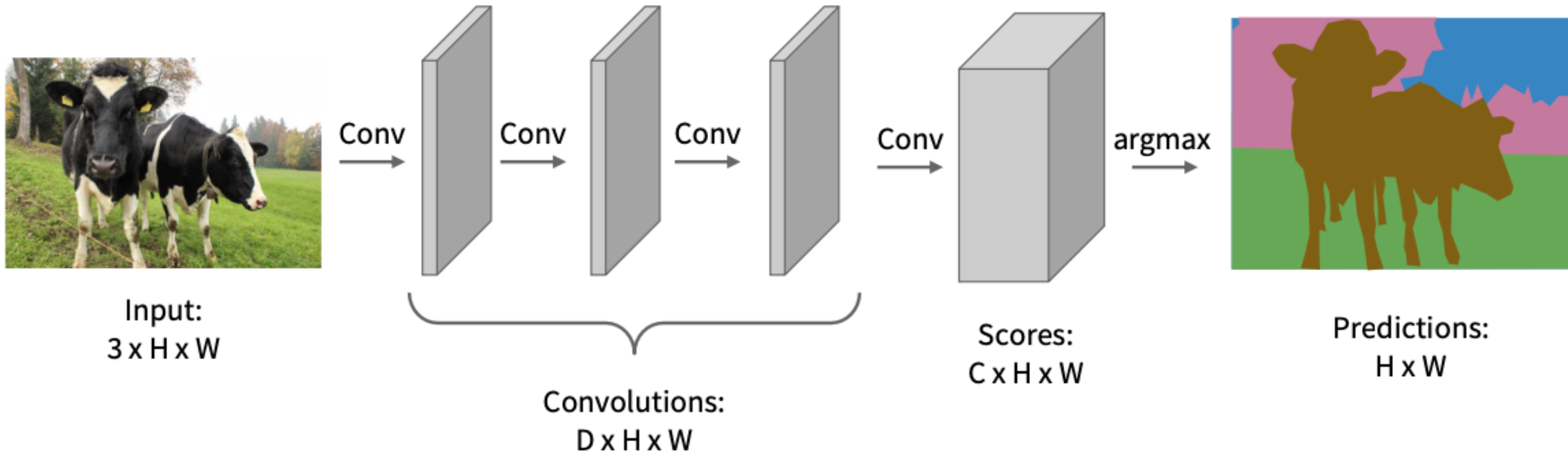


Cow

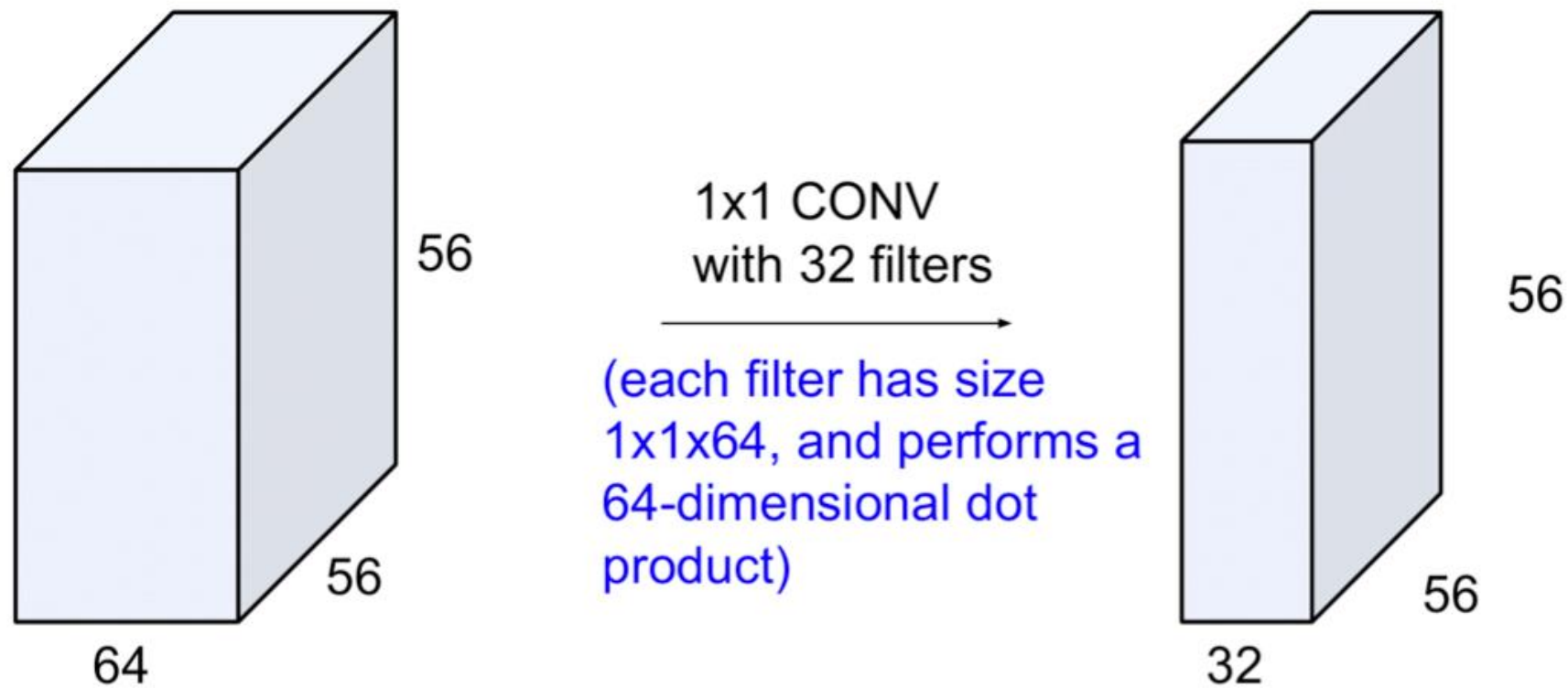


Grass

Подходы сегментации

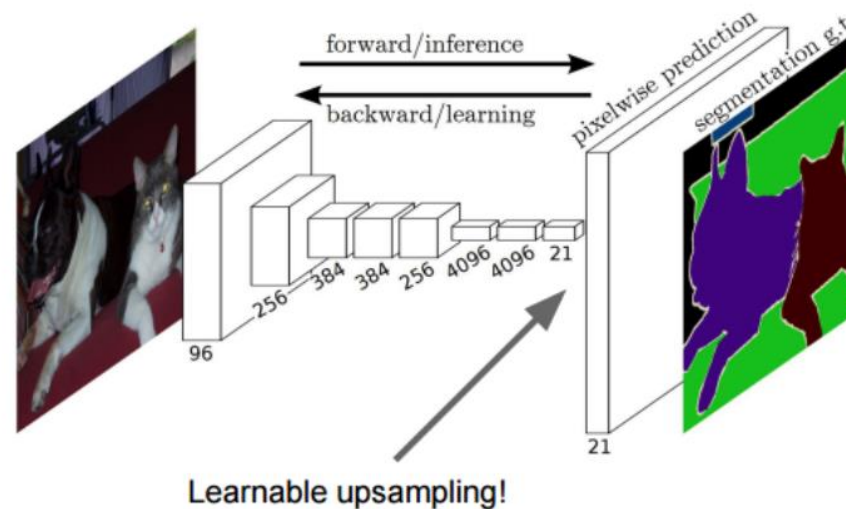
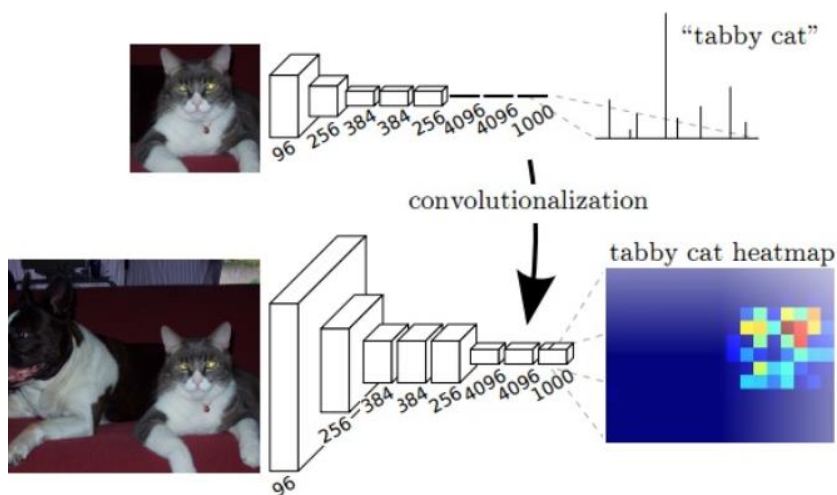


Conv1x1 Recap



Source: Stanford CS231n Lecture 5 2017 by Fei-Fei Li & Justin Johnson & Serena Yeung

Fully Convolutional Network: FCN



- Убираем FC и заменяем на Conv
- Добавляем декодер

Upsampling

Nearest neighbor

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

Bed of nails

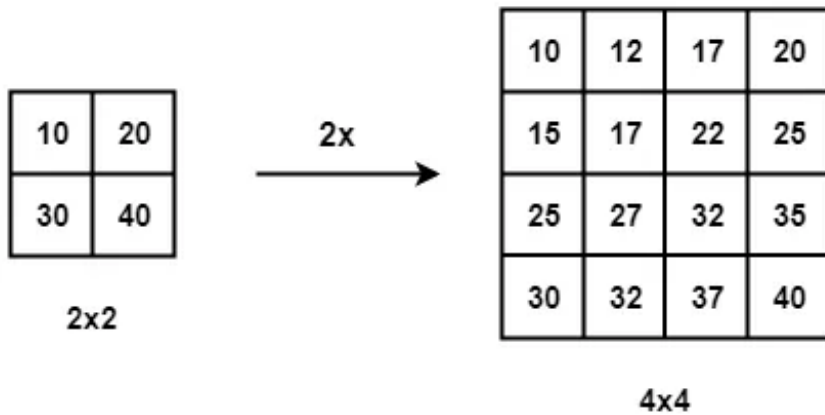
1	2
3	4



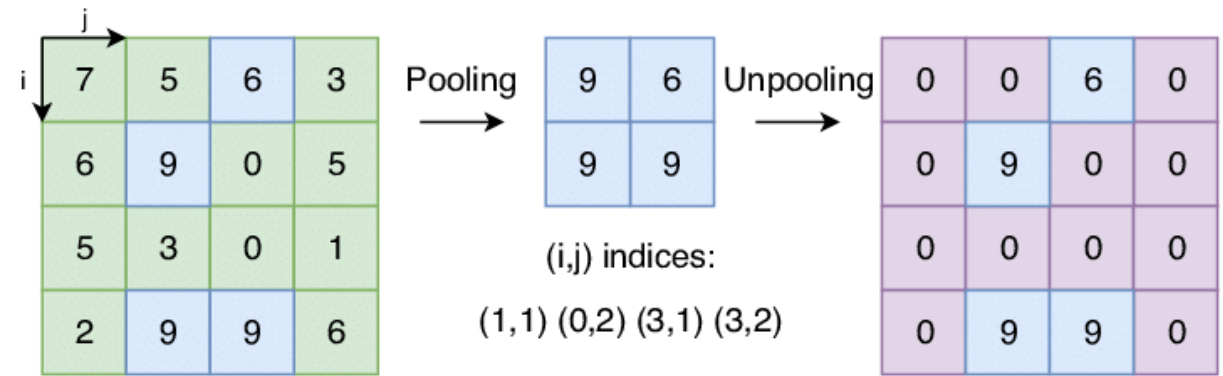
1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Upsampling

Bilinear interpolation

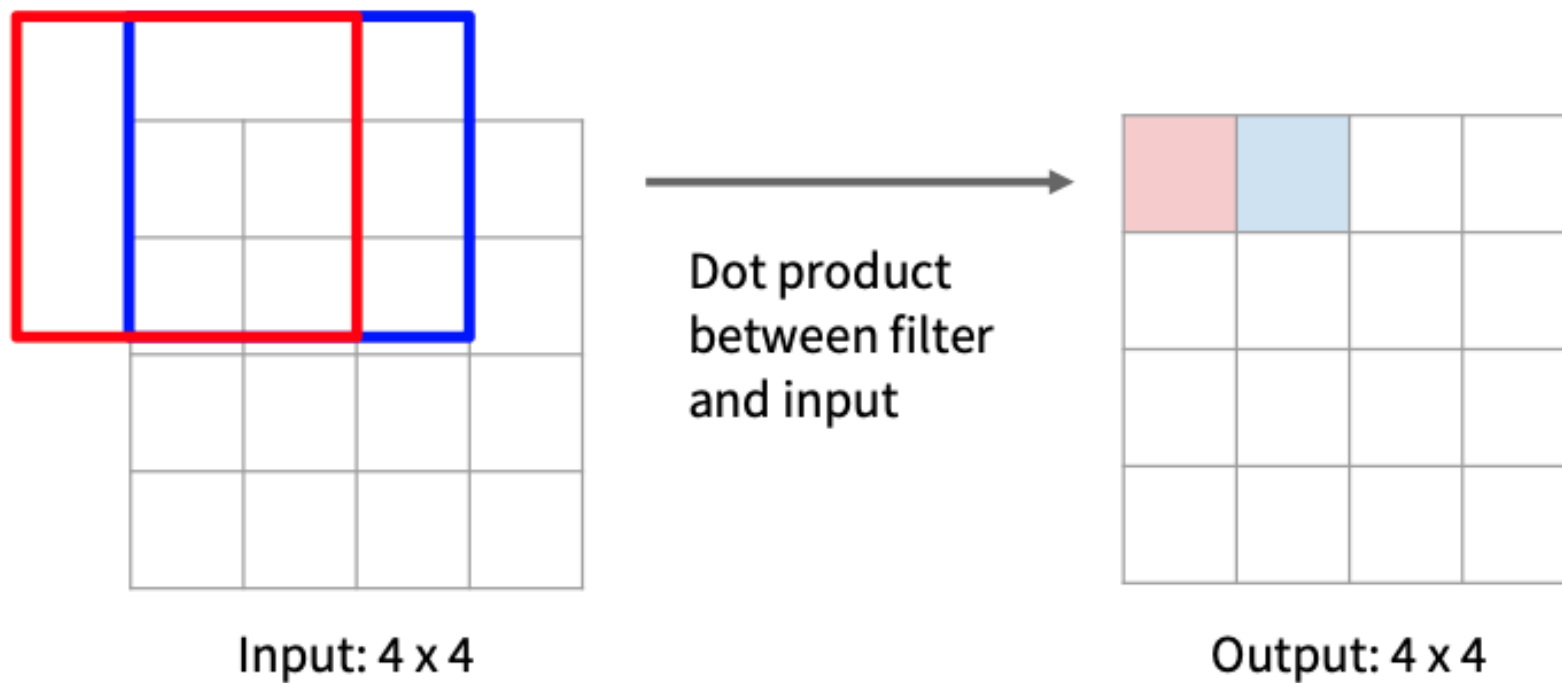


Max-Unpooling



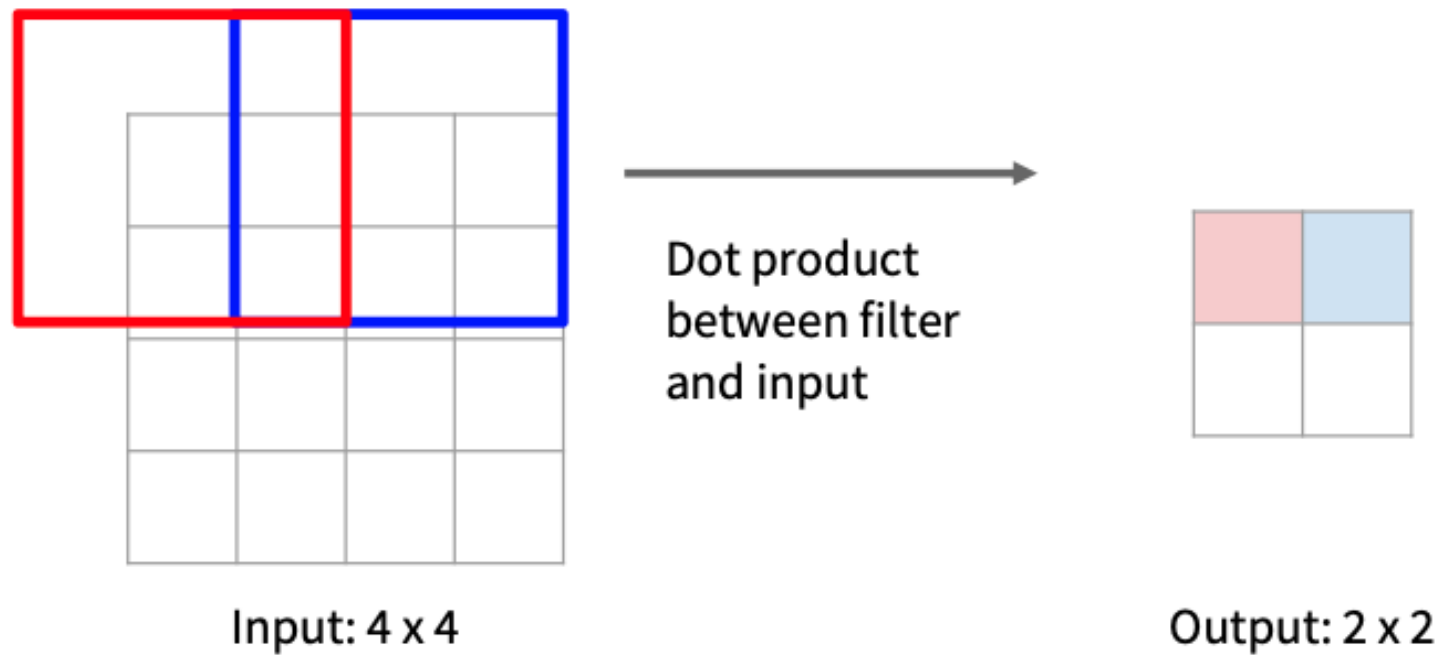
Upsampling

Вспомним: 3x3 Conv stride 1 pad 1



Upsampling

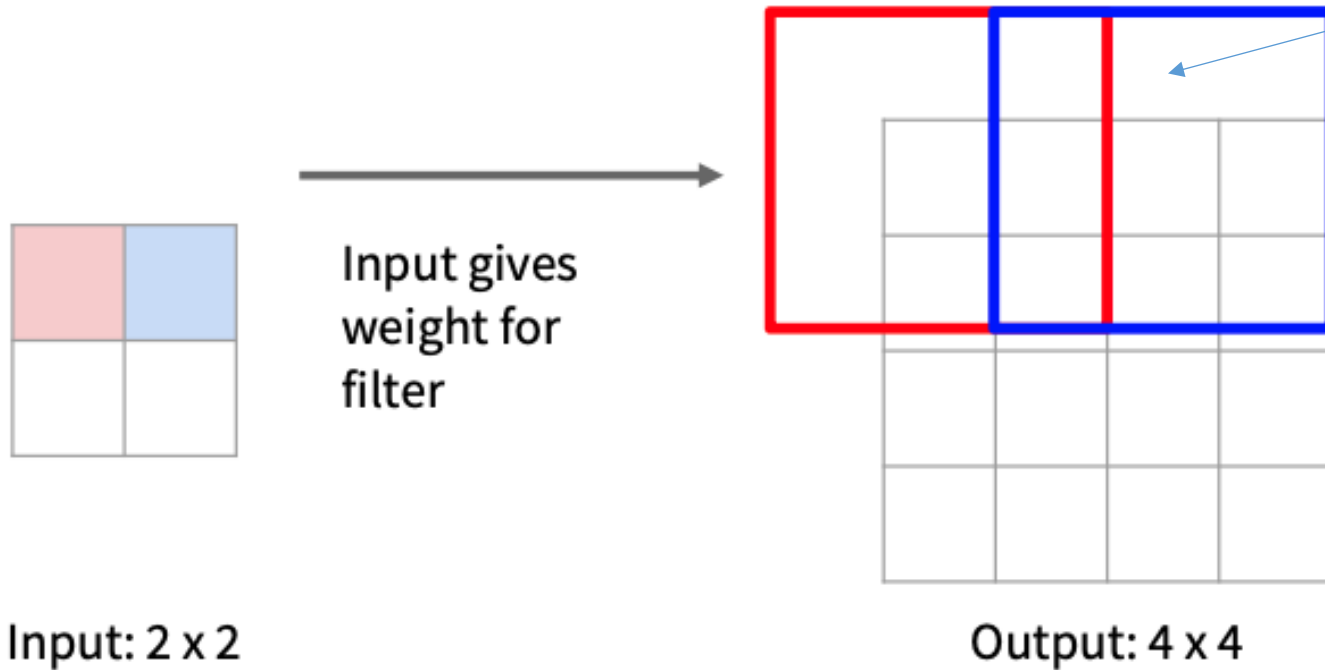
Вспомним: 3x3 Conv stride 2 pad 1



Upsampling

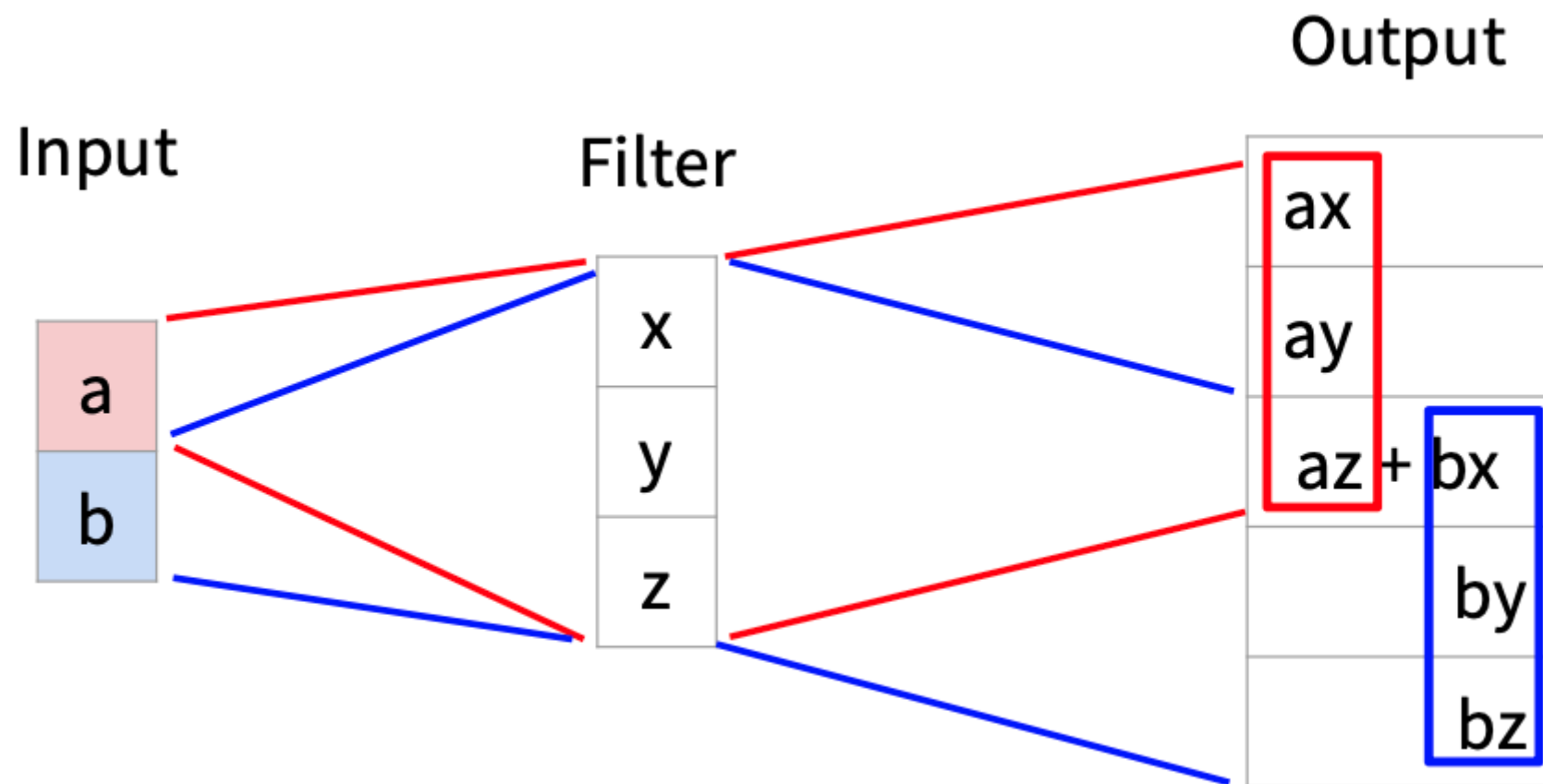
3 x 3 transposed convolution, stride 2 pad 1

Суммируем в области пересечения



Upsampling

Пример 1D



Upsampling

Представим свертки в виде уравнения – умножение матриц

свертки

Значения тензора

$$\vec{x} * \vec{a} = X \vec{a}$$

Матричное умножение (свертка)

kernel size

stride

padding

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

1D conv, kernel size=3,
stride=2, padding=1

Значения тензора

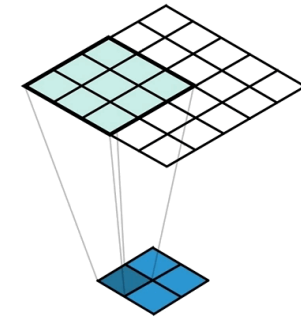
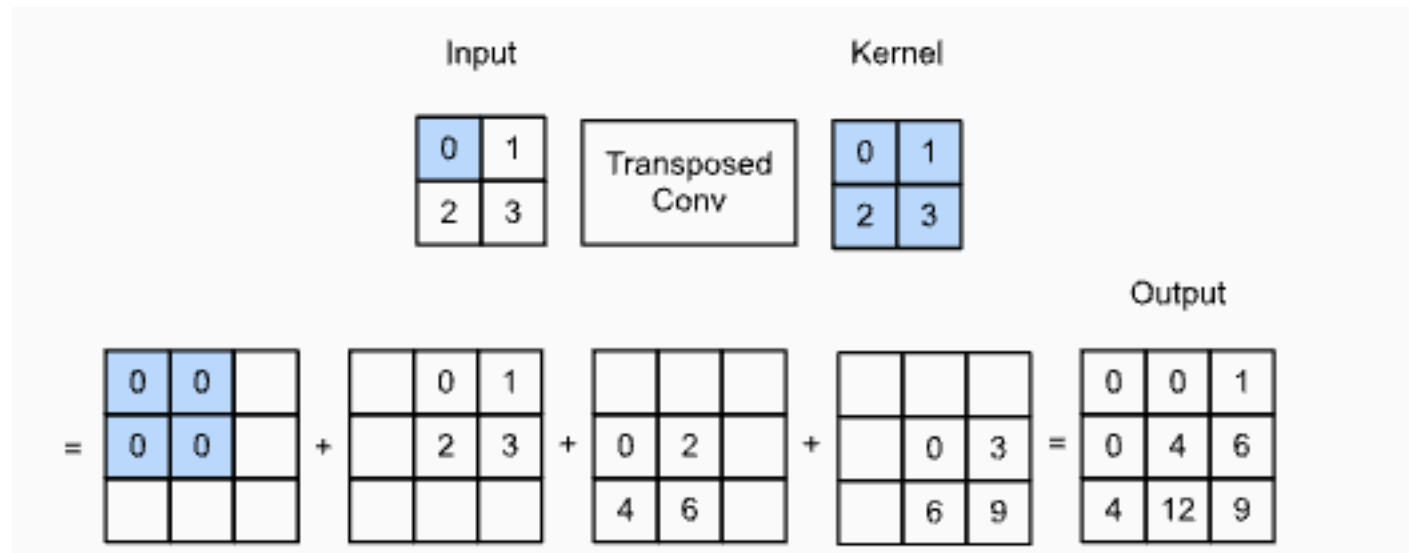
$$\vec{x} *^T \vec{a} = X^T \vec{a}$$

$$\begin{bmatrix} x & 0 \\ y & 0 \\ z & x \\ 0 & y \\ 0 & z \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax \\ ay \\ az + bx \\ by \\ bz \\ 0 \end{bmatrix}$$

Example: 1D transposed conv, kernel size=3,
stride=2, padding=0

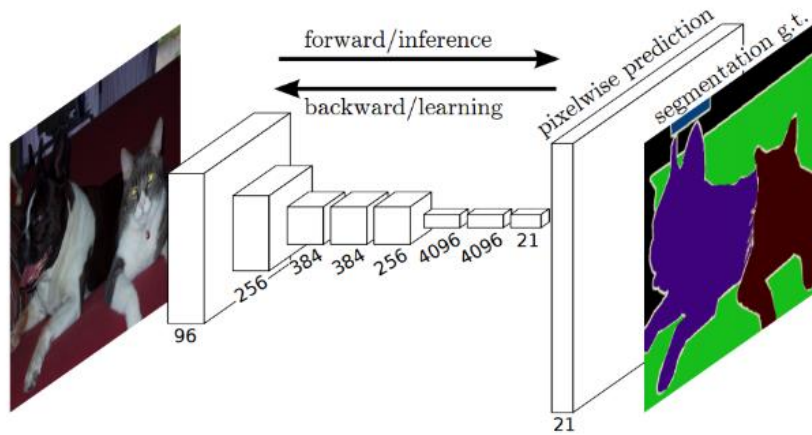
Upsampling

Transposed convolutions – also called fractionally strided convolutions – work by swapping the forward and backward passes of a convolution

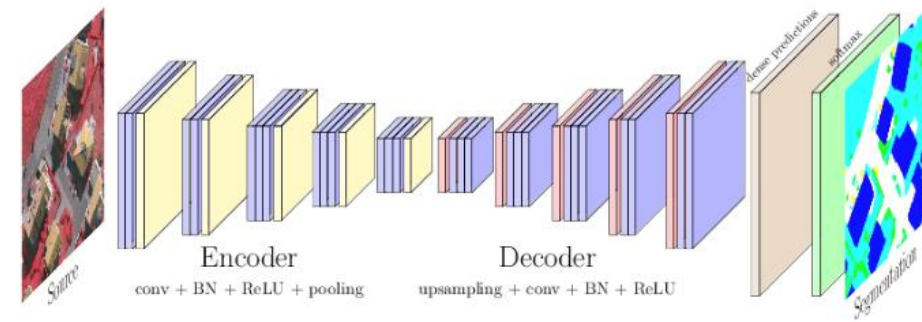


FCN8 to SegNet

FCN8



SegNet

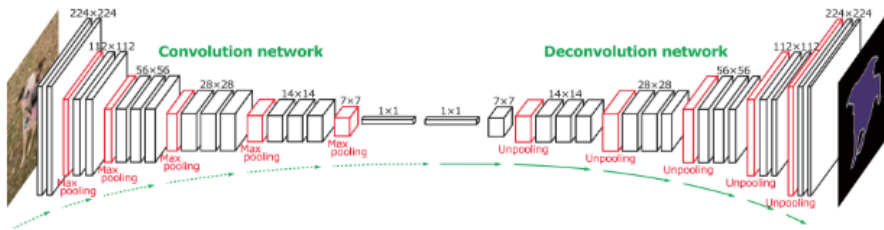


Заменить Upsampling на иерархический Upsampling

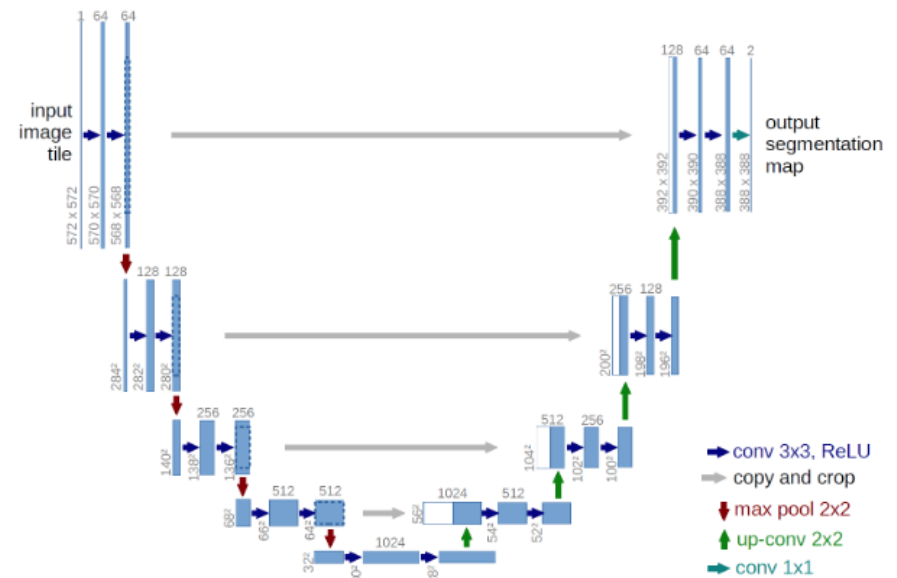
V. Badrinarayanan, A. Kendall, and R. Cipolla, "Segnet: A deep convolutional encoder-decoder architecture for image segmentation," arXiv:1511.00561, 2015

Upsampling

SegNet



UNet

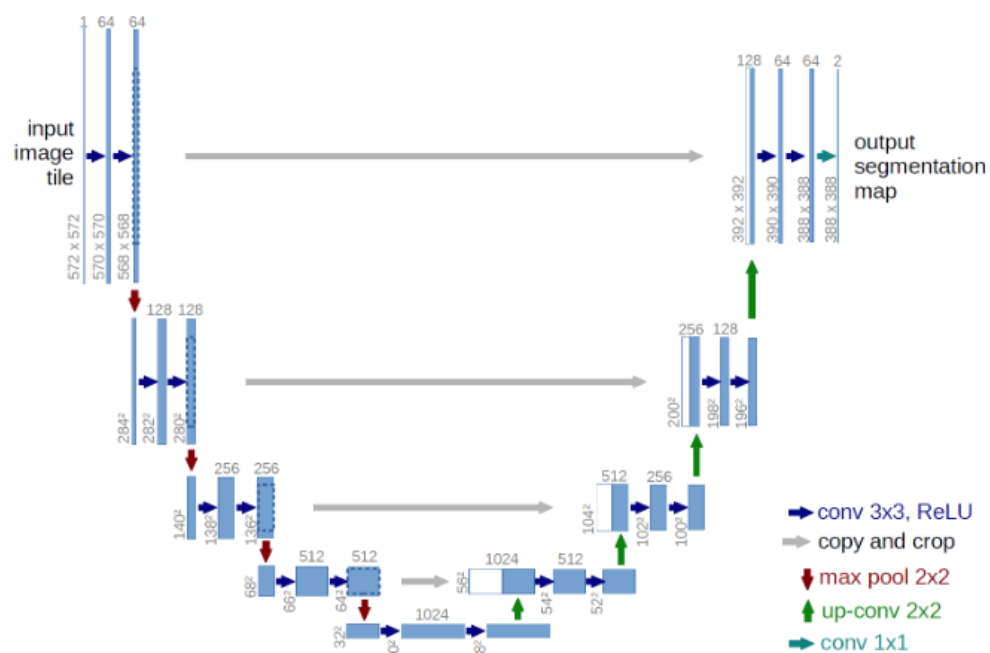


Added skip connections

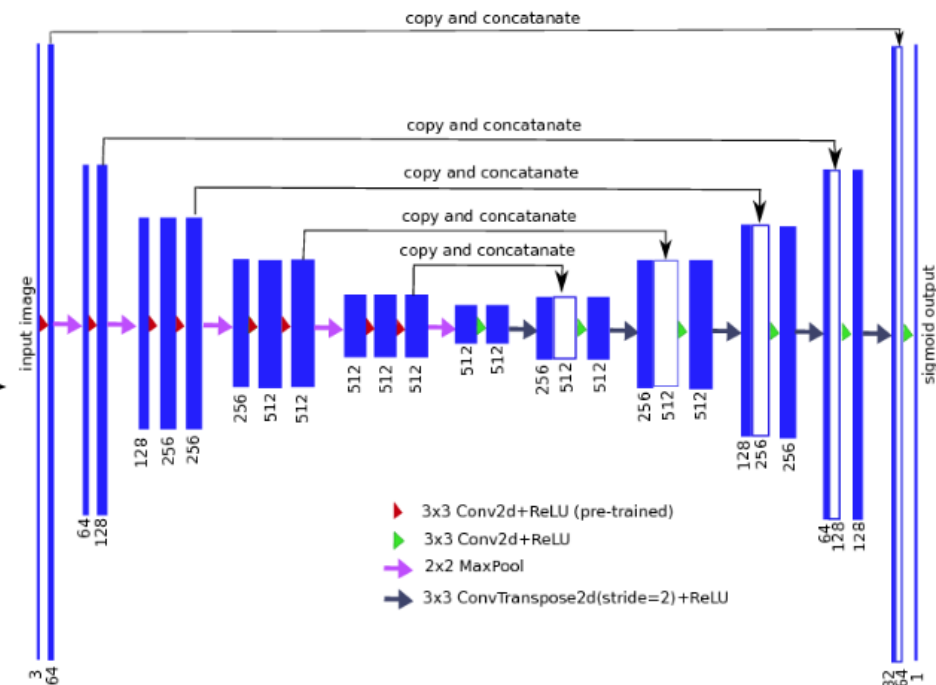
O. Ronneberger P. Fischer T. Brox "U-net: Convolutional networks for biomedical image segmentation" Proc. Med. Image Comput. Comput.-Assisted

Intervention pp. 234-241 2015.

Unet => TernausNet

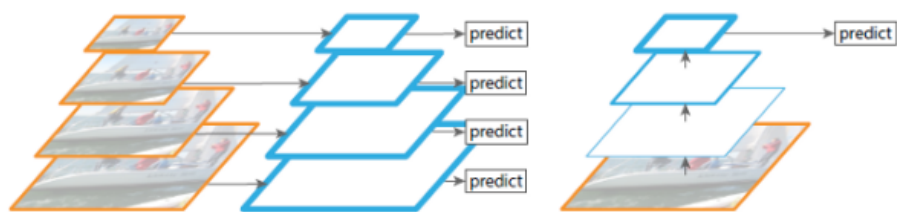


Text



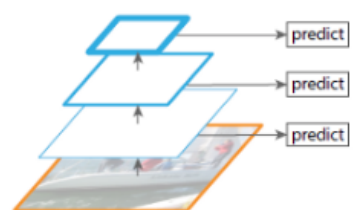
Энкодер инициализируем весами с ImageNet

Feature Pyramid Networks (FPN)

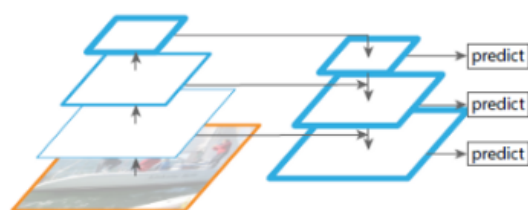


(a) Featurized image pyramid

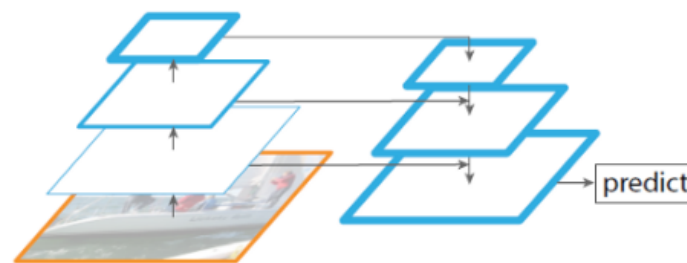
(b) Single feature map



(c) Pyramidal feature hierarchy



(d) Feature Pyramid Network



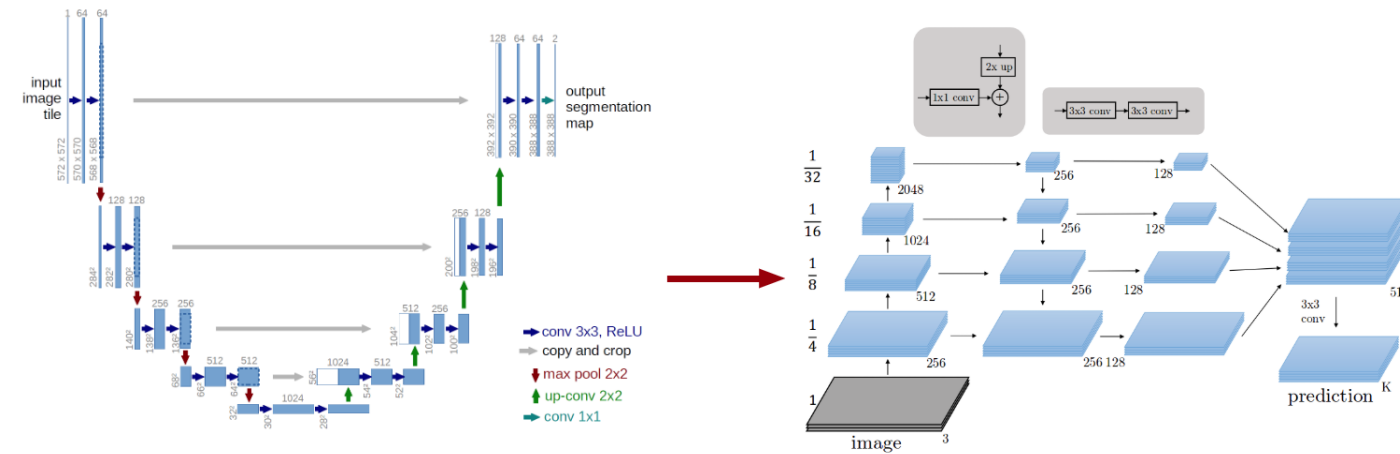
(e) Similar Structure with (d)

1. Легко добавить во многие архитектуры.
2. Помогает с multiscale

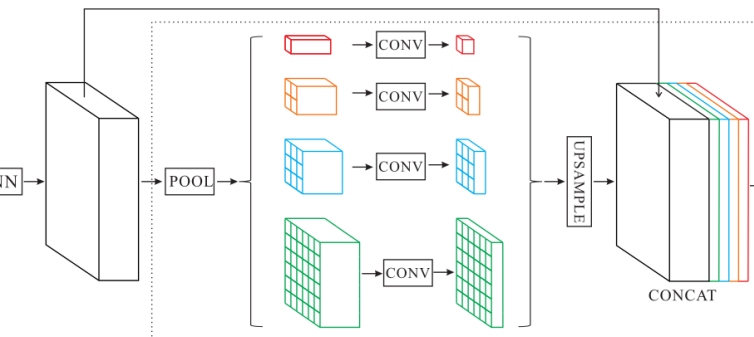
Tsung-Yi Lin, Piotr Dollar, Ross Girshick, Kaiming He, Bharath Hariharan, Serge Belongie; The IEEE Conference on Computer Vision and Pattern Recognition

(CVPR), 2017, pp. 2117-2125

Unet + FPN



(a) Input Image

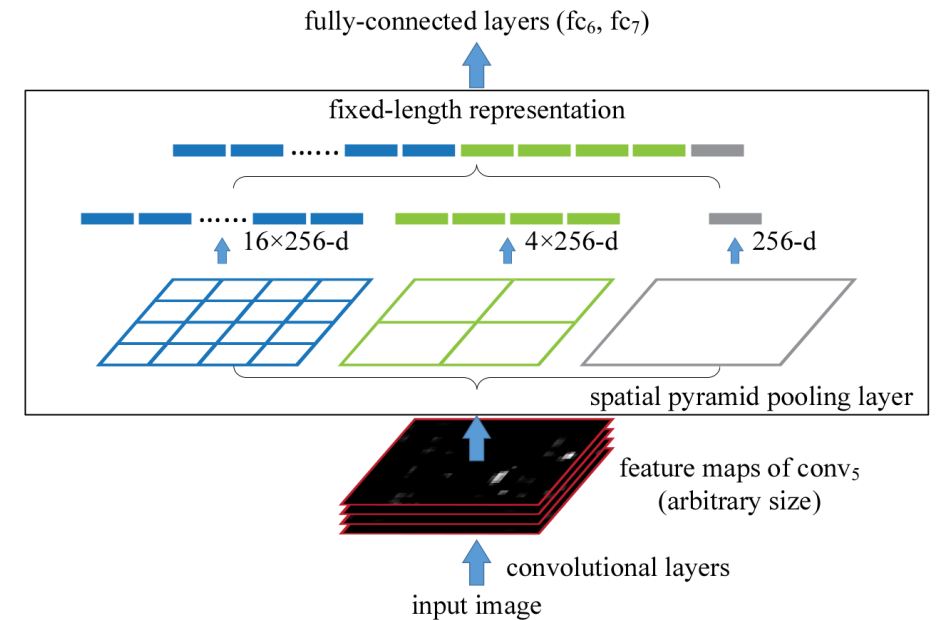


(b) Feature Map

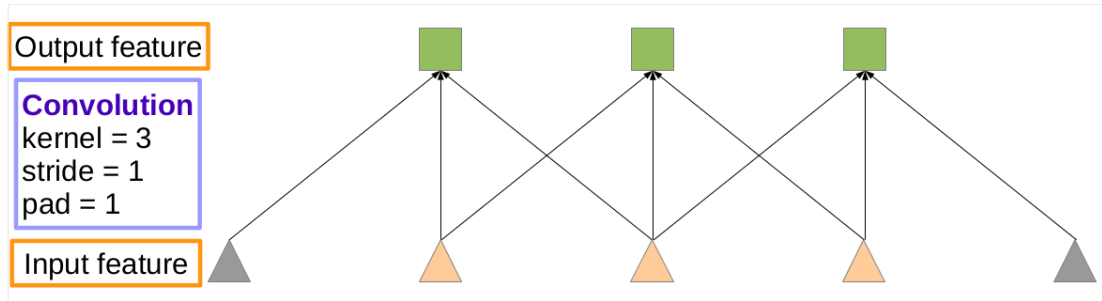
(c) Pyramid Pooling Module



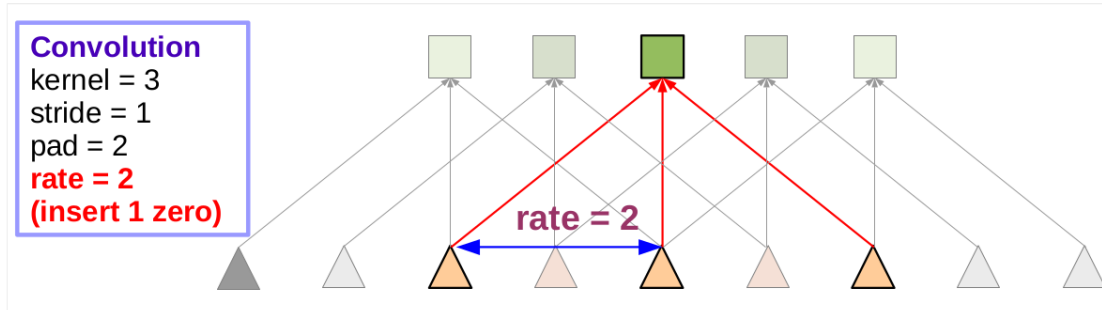
(d) Final Prediction



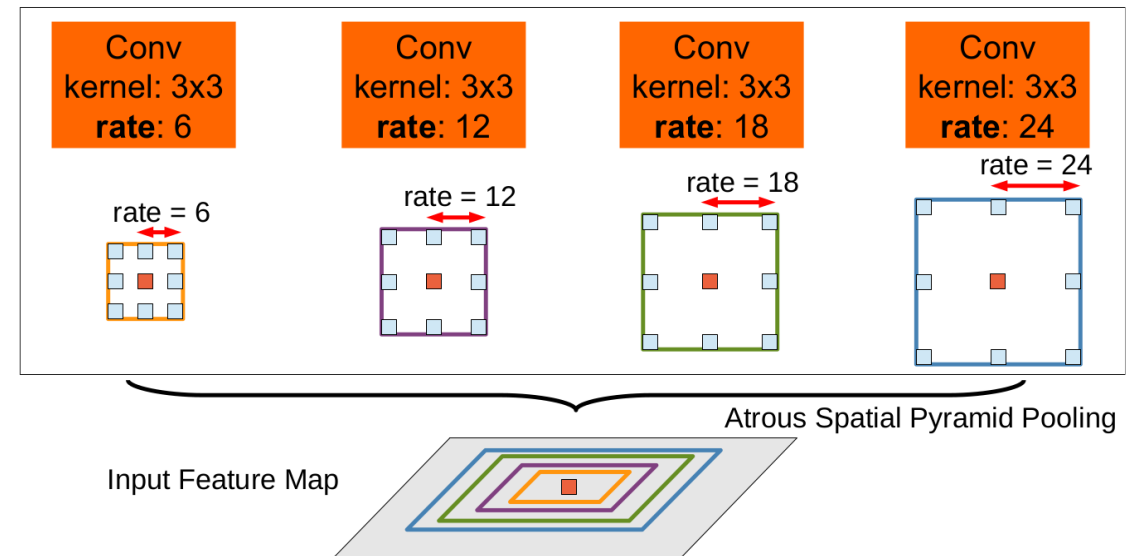
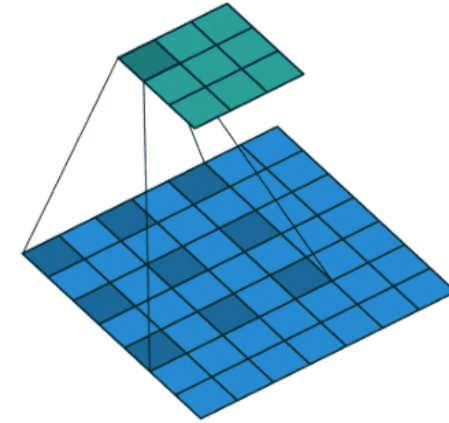
Atrous Spatial Pyramid Pooling



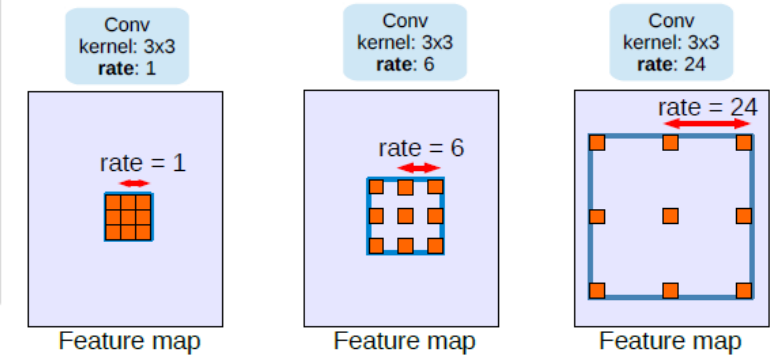
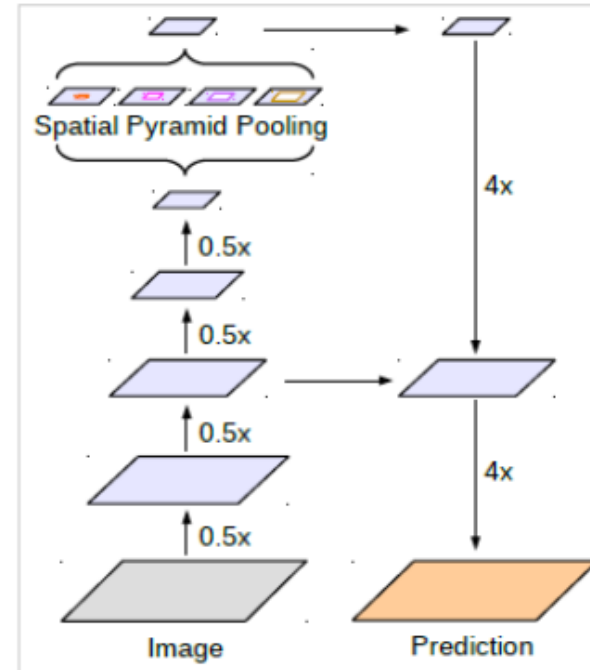
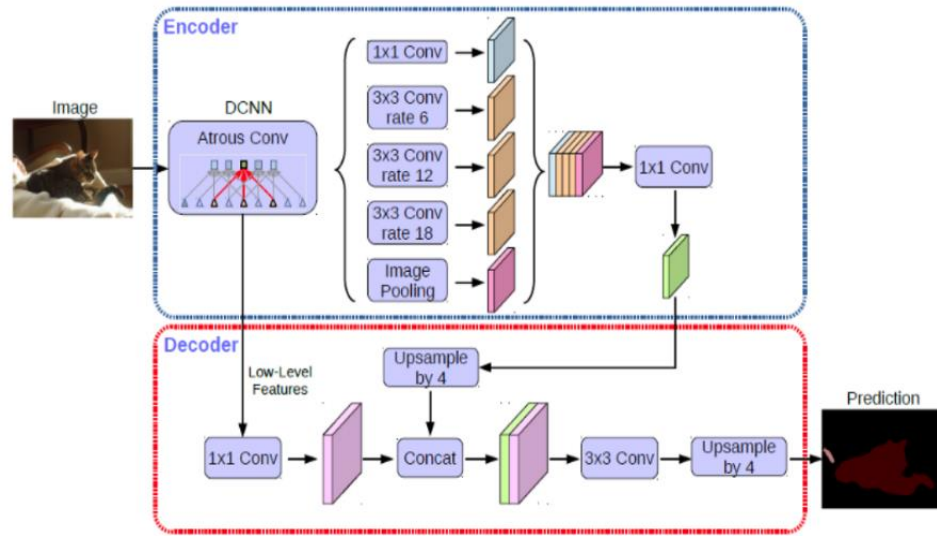
(a) Sparse feature extraction



(b) Dense feature extraction



DeepLabV3



Atrous Separable Convolution

Метрики

Intersection over Union (IOU) or Jaccard Index = $\frac{\text{Area of Overlap}}{\text{Area of Union}}$



Чаще всего используют
Dice - особенно в
медицинских снимках
и Jaccard (IoU)

Table 1. The three similarity coefficients

Similarity Coefficient (X,Y)	Actual Formula
Dice Coefficient	$2 \frac{ X \cap Y }{ X + Y }$
Cosine Coefficient	$\frac{ X \cap Y }{ X ^{1/2} \cdot Y ^{1/2}}$
Jaccard Coefficient	$\frac{ X \cap Y }{ X + Y - X \cap Y }$

Segmentation Loss Function

Каждый пиксель классификатор =>
Categorical / Binary Cross Entropy(CCE,
BCE)

Но! Метрика Dice / Jaccard

Dice / Jaccard недифференцируемы =>

Soft Dice / Soft Jaccard

и добавляем в loss

Lovasz-Softmax loss

Использовать для FineTune

$$CCE = \sum_c p(x) \log q(x)$$

$$LOSS = BCE - \ln(DICE)$$

$$BCE = - \sum_i (y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i))$$

$$DICE = 2 \frac{\sum_i y_i p_i}{\sum_u y_i + \sum p_i}$$

Berman, M., Rannen Triki, A., Blaschko, M.B.: The Lovász-softmax loss: a tractable surrogate for the optimization of the intersection-over-union measure in neural networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern

Recognition, pp. 4413–4421 (2018)

Label smoothing

Было

$$L = \sum_{i=1}^n H_i(p, q_\theta) \\ = - \sum_{i=1}^n \sum_{y=1}^K p(y|x_i) \log q_\theta(y|x_i)$$

Вместо использования вектора с одним активным битом мы вводим шумовое распределение $u(y|x)$ вида $(y|\theta)$. Наша новая истинная метка для данных (x_i, y_i) ($i=1, \dots, N$) будет

$$p'(y|x_i) = (1 - \varepsilon)p(y|x_i) + \varepsilon u(y|x_i) \\ = \begin{cases} 1 - \varepsilon + \varepsilon u(y|x_i) & \text{if } y = y_i \\ \varepsilon u(y|x_i) & \text{otherwise} \end{cases} \\ \varepsilon \in [0, 1] \quad \sum_{y=1}^K p'(y|x_i) = 1$$

Мы используем эту новую истинную метку вместо метки с одним активным битом в нашей функции потерь.

$$L' = - \sum_{i=1}^n \sum_{y=1}^K p'(y|x_i) \log q_\theta(y|x_i) \\ = - \sum_{i=1}^n \sum_{y=1}^K [(1 - \varepsilon)p(y|x_i) + \varepsilon u(y|x_i)] \log q_\theta(y|x_i)$$

$$L' = \sum_{i=1}^n \left\{ (1 - \varepsilon) \left[- \sum_{y=1}^K p(y|x_i) \log q_\theta(y|x_i) \right] + \varepsilon \left[- \sum_{y=1}^K u(y|x_i) \log q_\theta(y|x_i) \right] \right\} \\ = \sum_{i=1}^n \left[(1 - \varepsilon) H_i(p, q_\theta) + \varepsilon H_i(u, q_\theta) \right]$$

Как закодировать в обычном случае

$$y_k^{LS} = y_k(1 - \alpha) + \frac{\alpha}{K}$$

```
import torch
import torch.nn as nn

class LabelSmoothingLoss(nn.Module):
    def __init__(self, smoothing=0.0):
        super(LabelSmoothingLoss, self).__init__()
        self.smoothing = smoothing

    def forward(self, x, target):
        logprobs = torch.nn.functional.log_softmax(x, dim=-1)
        nll_loss = -logprobs.gather(dim=-1, index=target.unsqueeze(1)).squeeze(1)

        (1) smooth_loss = -logprobs.mean(dim=-1)
        loss = (1 - self.smoothing) * nll_loss + self.smoothing * smooth_loss
        return loss.mean()
```

Exponential Moving Average

Экспоненциальное скользящее среднее (ЕМА) - это техника, используемая в глубоком обучении, чтобы стабилизировать процесс обучения, сглаживая колебания в обучающих данных.

$$\text{EMA}_t = \begin{cases} p_1 & \text{if } t = 1 \\ \alpha p_t + (1 - \alpha) \text{EMA}_{t-1} & \text{else} \end{cases}$$

Псевдокод

```
# Initialize the EMA parameters
ema_alpha = 0.99
ema_weights = model.get_weights()

# Iterate through the training data
for i in range(num_epochs):
    for batch in training_data:
        # Perform forward and backward pass on the batch
        loss = model.train_on_batch(batch)

        # Update the model weights with stochastic gradient descent
        weights = model.get_weights()
        new_weights = [w - learning_rate * dw for w, dw in zip(weights, gradient
s)]

        # Update the EMA weights
        ema_weights = [ema_alpha * w + (1 - ema_alpha) * new_w for w, new_w in z
ip(ema_weights, new_weights)]

        # Set the model weights to the EMA weights
        model.set_weights(ema_weights)
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.swa_utils import AveragedModel, SWALR

# define your neural network
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(784, 100)
        self.fc2 = nn.Linear(100, 10)

    def forward(self, x):
        x = x.view(-1, 784)
        x = nn.functional.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# define your optimizer
optimizer = optim.SGD(net.parameters(), lr=0.01)

# define the EMA
ema_model = AveragedModel(net)

# define the SWA learning rate scheduler
swa_scheduler = SWALR(optimizer, swa_lr=0.05)

# train your model
for epoch in range(num_epochs):
    for i, (inputs, labels) in enumerate(train_loader):
        optimizer.zero_grad()
        outputs = net(inputs)
        loss = loss_function(outputs, labels)
        loss.backward()
        optimizer.step()
        ema_model.update_parameters(net)
        swa_scheduler.step()

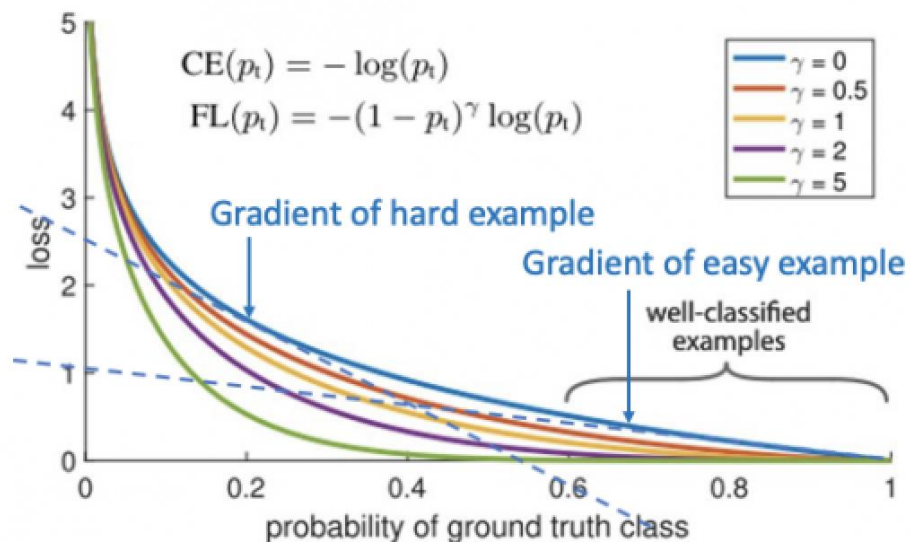
# evaluate your model using the EMA
ema_model.eval()
with torch.no_grad():
    for inputs, labels in test_loader:
        outputs = ema_model(inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
accuracy = 100 * correct / total
```

Focal Loss

Фокальная функция потерь - это функция потерь, разработанная для задач обнаружения объектов в глубоком обучении.

Она разработана для решения проблемы несбалансированности классов, когда количество примеров в одном классе значительно превышает количество примеров в других классах.

$$FL(p_t) = -(1 - p_t)^\gamma \log(p_t)$$



```
def focal_loss(y_true, y_pred, alpha=0.25, gamma=2.0):  
    # Convert predictions to probabilities  
    y_pred = K.softmax(y_pred, axis=-1)  
    # Calculate the cross entropy loss  
    ce_loss = -y_true * K.log(y_pred)  
    # Calculate the focal loss  
    pt = K.sum(y_true * y_pred, axis=-1)  
    fl_loss = alpha * K.pow(1 - pt, gamma) * ce_loss  
    return K.mean(fl_loss)
```

Заключение

Рассмотрена постановка задачи сегментации изображений

Изучены базовые подходы архитектур encoder-decoder

- pooling

- upconv, upsampling

- unet

Рассмотрены техники обучения моделей

- Label smoothing

- EMA

- Focal loss

Ссылки

Kaggle - [Airbus Ship Detection Challenge](#), пример на задачу instance segmentation